AI Assisted Coding

Name : Vineeth-Chidurala **Date** : 13-03-2026

Ht.No. : 2303A52447

Task - 1 : Sorting Student Records for Placement Drive
SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order

Prompt : Generate a Java program to manage student records for a university placement drive.

Requirements:

1. Create a Student class with fields: name, rollNumber, and cgpa.

2. Store multiple student records in an ArrayList.

3. Implement two sorting algorithms to sort students by CGPA in descending order:

- Quick Sort
- Merge Sort

4. Measure the runtime performance of both algorithms using System.nanoTime().

5. Create a function to display the top 10 students with the highest CGPA.

6. Generate a large dataset of students (for example 10000 records) to test performance.

7. Print:

- Execution time for Quick Sort
- Execution time for Merge Sort
- Top 10 students based on CGPA.

Screenshots :

```python
# Generate a Java program to manage student records for a university placement drive.
# Requirements:
# 1. Create a Student class with fields: name, rollNumber, and cgpa.
# 2. Store multiple student records in an ArrayList.
# 3. Implement two sorting algorithms to sort students by CGPA in descending order:
#    - Quick Sort
#    - Merge Sort
# 4. Measure the runtime performance of both algorithms using System.nanoTime().
# 5. Create a function to display the top 10 students with the highest CGPA.
# 6. Generate a large dataset of students (for example 10000 records) to test performance.
# 7. Print:
#    - Execution time for Quick Sort
#    - Execution time for Merge Sort
#    - Top 10 students based on CGPA.

import random
import time

# Student class
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"{self.roll_number} - {self.name} : {self.cgpa:.2f}"


# Generate random students
def generate_students(n):
```

Output :

```python
merge_sort(students_merge)
end = time.time()

merge_time = end - start


print("Quick Sort Time:", quick_time, "seconds")
print("Merge Sort Time:", merge_time, "seconds")


display_top_10(students_quick)
```

[1] ✓ 0.0s

```
Quick Sort Time: 0.023060321807861328 seconds
Merge Sort Time: 0.029453754425048828 seconds

Top 10 Students Based on CGPA:

1. 8917 - Student8917 : 10.00
2. 925 - Student925 : 10.00
3. 1160 - Student1160 : 10.00
4. 2401 - Student2401 : 10.00
5. 3392 - Student3392 : 10.00
6. 3975 - Student3975 : 10.00
7. 4488 - Student4488 : 10.00
8. 4676 - Student4676 : 10.00
9. 4957 - Student4957 : 10.00
10. 5632 - Student5632 : 10.00
```

Task – 2 : Implementing Bubble Sort with AI Comments

Write a Python implementation of Bubble Sort.

Prompt : Write a Python program that implements the Bubble Sort algorithm.

Requirements:

1. Sort a list of numbers in ascending order using Bubble Sort.
2. Add clear inline comments explaining the key logic:

    - comparison between elements
    - swapping elements
    - passes through the list
    - stopping condition
3. Print the original list and the sorted list.
4. Provide a short time complexity analysis of Bubble Sort.
5. Keep the code simple and readable for students.

ScreenShots :

```python
# Write a Python program that implements the Bubble Sort algorithm.

# Requirements:

# 1. Sort a list of numbers in ascending order using Bubble Sort.
# 2. Add clear inline comments explaining the key logic:

#    * comparison between elements
#    * swapping elements
#    * passes through the list
#    * stopping condition
# 3. Print the original list and the sorted list.
# 4. Provide a short time complexity analysis of Bubble Sort.
# 5. Keep the code simple and readable for students.

def bubble_sort(arr):

    n = len(arr)

    for i in range(n):

        swapped = False  # Flag to check if any swapping happens in this pass

        for j in range(0, n - i - 1):

            if arr[j] > arr[j + 1]:

                arr[j], arr[j + 1] = arr[j + 1], arr[j]

                swapped = True  # Mark that a swap occurred

        if not swapped:
            break
```

Output :

```python
print("Original List:", numbers)

# Call Bubble Sort
bubble_sort(numbers)

print("Sorted List:", numbers)
```

[2] ✓ 0.0s

```
Original List: [64, 34, 25, 12, 22, 11, 90]
Sorted List: [11, 12, 22, 25, 34, 64, 90]
```

Task-3 : Quick Sort and Merge Sort Comparison

Implement Quick Sort and Merge Sort using recursion.

Prompt : Complete the missing logic for Quick Sort and Merge Sort in Python using recursion.

Requirements:

1. Implement Quick Sort and Merge Sort algorithms using recursion.
2. Add proper Python docstrings explaining each function.
3. Use randomly generated lists of numbers.
4. Compare the performance of both algorithms on:
    - Random list
    - Already sorted list
    - Reverse sorted list
5. Measure runtime using the time module.
6. Print the results in a readable format.

ScreenShots :

```python
# Complete the missing logic for Quick Sort and Merge Sort in Python using recursion.

# Requirements:

# 1. Implement Quick Sort and Merge Sort algorithms using recursion.
# 2. Add proper Python docstrings explaining each function.
# 3. Use randomly generated lists of numbers.
# 4. Compare the performance of both algorithms on:

#    * Random list
#    * Already sorted list
#    * Reverse sorted list
# 5. Measure runtime using the time module.
# 6. Print the results in a readable format.

import random
import time


def quick_sort(arr):
    """
    Sorts a list using the Quick Sort algorithm with recursion.

    Quick Sort works by selecting a pivot element and partitioning
    the list into elements smaller than the pivot and greater than the pivot.
    The same process is recursively applied to the sublists.
    """

    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]

    left = [x for x in arr if x < pivot]
```

Output :

```python
print("\nReverse Sorted List:")
print("Quick Sort:", measure_time(quick_sort, reverse_list))
print("Merge Sort:", measure_time(merge_sort, reverse_list))
```

[3] ✓ 0.0s

```
Performance Comparison

Random List:
Quick Sort: 0.0122528076171875
Merge Sort: 0.02101755142211914

Sorted List:
Quick Sort: 0.0076749324798583984
Merge Sort: 0.008596181869506836

Reverse Sorted List:
Quick Sort: 0.005189657211303711
Merge Sort: 0.007564067840576172
```

<u>Task-4</u> : Real-Time Application – Inventory Management System

A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

<u>Prompt</u> : Design an inventory management system in Python for a retail store.

Requirements:

1. Each product should have product_id, name, price, and quantity.
2. Recommend efficient algorithms for:
    - Searching a product by ID
    - Searching a product by name
    - Sorting products by price
    - Sorting products by quantity
3. Implement the recommended algorithms in Python.
4. Provide a table explaining:

   operation → algorithm → justification.
5. Use realistic inventory data and demonstrate the functions.

Screenshots :

```python
# Design an inventory management system in Python for a retail store.
# Requirements:
# 1. Each product should have product_id, name, price, and quantity.
# 2. Recommend efficient algorithms for:
#    * Searching a product by ID
#    * Searching a product by name
#    * Sorting products by price
#    * Sorting products by quantity
# 3. Implement the recommended algorithms in Python.
# 4. Provide a table explaining:
#    operation → algorithm → justification.
# 5. Use realistic inventory data and demonstrate the functions.

import random

# Product class
class Product:
    def __init__(self, product_id, name, price, quantity):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.quantity = quantity

    def __repr__(self):
        return f"{self.product_id} | {self.name} | Price: {self.price} | Qty: {self.quantity}"


# --------------- SEARCH FUNCTIONS ---------------

def search_by_id(inventory_dict, product_id):
    """
    Search product using dictionary (hash table).
    Time Complexity: O(1) average
    """
```

Output :

```python
print("\nSorted by Price (Merge Sort):")
sorted_price = merge_sort(products)
for p in sorted_price:
    print(p)


print("\nSorted by Quantity (Quick Sort):")
sorted_quantity = quick_sort(products)
for p in sorted_quantity:
    print(p)
```

[4] ✓ 0.0s

```
Search by ID:
103 | Keyboard | Price: 1500 | Qty: 60

Search by Name:
104 | Monitor | Price: 12000 | Qty: 25

Sorted by Price (Merge Sort):
102 | Mouse | Price: 500 | Qty: 120
103 | Keyboard | Price: 1500 | Qty: 60
105 | Headphones | Price: 2000 | Qty: 40
104 | Monitor | Price: 12000 | Qty: 25
101 | Laptop | Price: 80000 | Qty: 15

Sorted by Quantity (Quick Sort):
101 | Laptop | Price: 80000 | Qty: 15
104 | Monitor | Price: 12000 | Qty: 25
105 | Headphones | Price: 2000 | Qty: 40
103 | Keyboard | Price: 1500 | Qty: 60
102 | Mouse | Price: 500 | Qty: 120
```

Task-5 : Real-Time Stock Data Sorting & Searching

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

Prompt : Create a Python program that simulates a real-time stock data analysis system.
Requirements:
Simulate stock data containing stock symbol, opening price, and closing price.
Calculate percentage change for each stock.
Implement Heap Sort to rank stocks based on percentage gain/loss.
Implement a search system using Hash Maps (Python dictionary) to instantly retrieve stock information by symbol.
Compare performance with:
Python sorted() function
Dictionary lookup
Print sorted stock rankings and allow searching by stock symbol.
Provide performance comparison results and analysis.

Screenshots :

```python
# Create a Python program that simulates a real-time stock data analysis system.

# Requirements:

# 1. Simulate stock data containing stock symbol, opening price, and closing price.
# 2. Calculate percentage change for each stock.
# 3. Implement Heap Sort to rank stocks based on percentage gain/loss.
# 4. Implement a search system using Hash Maps (Python dictionary) to instantly retrieve stock information by symbol.
# 5. Compare performance with:

#    * Python sorted() function
#    * Dictionary lookup
# 6. Print sorted stock rankings and allow searching by stock symbol.
# 7. Provide performance comparison results and analysis.

import random
import time
# ---------------- STOCK CLASS ----------------
class Stock:
    def __init__(self, symbol, open_price, close_price):
        self.symbol = symbol
        self.open_price = open_price
        self.close_price = close_price
        self.change = ((close_price - open_price) / open_price) * 100
    def __repr__(self):
        return f"{self.symbol} | Open:{self.open_price} | Close:{self.close_price} | Change:{self.change:.2f}%"
# ---------------- GENERATE STOCK DATA ----------------
def generate_stocks(n):
    stocks = []
    for i in range(n):
        symbol = f"STK{i}"
        open_price = round(random.uniform(100, 1000), 2)
        close_price = round(random.uniform(100, 1000), 2)
        stocks.append(Stock(symbol, open_price, close_price))
```

Output :

```python
print("Heap Sort Time:", end - start)


# Python sorted() performance
start = time.time()
sorted_data = sorted(stocks, key=lambda x: x.change)
end = time.time()

print("Built-in sorted() Time:", end - start)


# Display Top 5 gainers
print("\nTop 5 Stocks by % Change:")
for stock in sorted_data[-5:]:
    print(stock)


# Search example
symbol = "STK10"
print("\nSearch Result:")
print(search_stock(stock_dict, symbol))
```

[5] ✓ 0.0s

```
Heap Sort Time: 0.0049343109130859375
Built-in sorted() Time: 0.0009315013885498047

Top 5 Stocks by % Change:
STK590 | Open:116.57 | Close:871.76 | Change:647.84%
STK829 | Open:111.63 | Close:849.24 | Change:660.76%
STK776 | Open:122.16 | Close:935.23 | Change:665.58%
STK759 | Open:127.48 | Close:977.38 | Change:666.69%
STK544 | Open:106.68 | Close:829.97 | Change:678.00%

Search Result:
STK10 | Open:780.84 | Close:157.68 | Change:-79.81%
```